

VULNERABILITY ASSESSMENT & PENETRATION TESTING REPORT

Security Evaluation of Target Infrastructure

Target Application:	OWASP Juice Shop [cite: 1]
Testing Tool:	OWASP ZAP (Zaproxy) [cite: 1]
Prepared By:	Mariam Shahzadi [cite: 1]

1. Executive Summary

The primary objective of this assessment was to evaluate the security posture of the OWASP Juice Shop web application deployed in a local, controlled testing environment. OWASP Juice Shop is an intentionally insecure web application designed for security training and testing [cite: 1].

An automated vulnerability assessment was conducted using OWASP ZAP (Zaproxy), an industry-standard web application security scanner. The assessment simulated real-world attack vectors to identify security loopholes, misconfigurations, and potential data exposure risks. Multiple vulnerabilities ranging from high to low severity were identified, indicating that the application in its current state is highly susceptible to unauthorized access and data breaches. Immediate remediation is required before any similar code architectures are deployed in a live production environment [cite: 1].

2. Project Scope & Environment Setup

2.1 Target Information

- **Application Name:** OWASP Juice Shop [cite: 1]
- **Target Environment:** Localhost (Linux Subsystem) [cite: 1]
- **Target URL:** `http://localhost:3000` [cite: 1]

2.2 Methodology & Execution Steps

The installation, deployment, and testing phase followed a structured, sequential methodology to guarantee a repeatable testing framework [cite: 1].

- 1. Environment Updating & Node Package Setup (Phase 1: Dependencies):** The local system repository was updated to fetch the latest package lists. Following this, Node.js and NPM (Node Package Manager) were installed. Node.js provides the runtime environment required to execute the JavaScript-based Juice Shop backend server [cite: 1].

```
sudo apt update
sudo apt install nodejs npm
nodejs -v
npm -v
```

- 2. Application Source Code Deployment (Phase 2: Cloning & Launch):** The official source code of the OWASP Juice Shop repository was cloned from GitHub to the local machine. After navigating into the project folder, all backend and frontend dependencies specified in package.json were downloaded and compiled [cite: 1].

```
git clone https://github.com/juice-shop/juice-shop.git
cd juice-shop
npm install
npm start
```

The application listener initialized successfully on default port 3000 [cite: 1].

3. **Security Scanner Integration (Phase 3: Attack Setup):** OWASP ZAP (Zaproxy) was installed directly via the Advanced Package Tool (APT) manager. ZAP acts as a man-in-the-middle proxy, intercepting and inspecting traffic between the browser and the target web app to discover underlying application flaws [cite: 1].

```
sudo apt update && sudo apt install zaproxy -y
zaproxy
```

3. Assessment Methodology & Scan Process

The penetration testing phase utilizing OWASP ZAP was divided into two distinct technical stages [cite: 1]:

3.1 Passive Scanning & Spidering (Passive Reconnaissance)

Once the application URL (`http://localhost:3000`) was fed into OWASP ZAP, the Traditional Spider tool was launched. The spider mapped the entire web application by crawling links, parsing HTML structures, and uncovering hidden endpoints [cite: 1].

Mechanism: Passive scanning evaluates requests and responses passing through the system without modifying the data packets. It identifies architectural flaws like missing security headers, exposed cookies, or plain-text credentials without alerting or disrupting the application ecosystem [cite: 1].

3.2 Active Scanning (Attacking Phase)

Following the spidering phase, an Active Scan was initiated against the mapped site structure [cite: 1].

Mechanism: Unlike passive scanning, active scanning actively constructs and sends malicious, modified payloads to the application endpoints. It simulates real hacker behavior by injecting scripts, trying SQL syntaxes, and forcing buffer overflows to see if the web server drops unhandled errors or exposes raw database components [cite: 1].

4. Vulnerability Findings & Analysis

Severity Level	Affected Endpoint / URL	Vulnerability Discovered	Technical Description
High	<code>http://localhost:3000/rest/user/login</code>	SQL Injection (SQLi)	The application accepts user inputs directly inside database queries without input sanitization or parameterization. Attackers can bypass authentication forms [cite: 1].
Medium	<code>http://localhost:3000/#/search?q=</code>	Cross-Site Scripting (XSS)	User input provided in the search parameter is executed directly by the web browser. Attackers can leverage this to steal session tokens or inject malicious scripts [cite: 1].
Low	Multiple Form Endpoints	Absence of Anti-CSRF Tokens	Forms do not incorporate unique, random cryptographic tokens. This leaves users vulnerable to Cross-Site Request Forgery [cite: 1].
Info	All Pages (HTTP Response)	Missing Security Headers	Essential headers such as X-Content-Type-Options, X-Frame-Options, and Strict-Transport-Security are absent [cite: 1].

Admin Login Success Proof

Figure 1: Successful Administrative Login via SQL Injection Bypass

Login Form Payload Injection

Figure 2: Injecting Payload in the Login Endpoint

XSS Pop-up Execution

Figure 3: XSS Alert Execution in Search Bar

OWASP ZAP Active Scan Results

Figure 4: OWASP ZAP Active Scan & Alerts Dashboard

5. Detailed Technical Breakdown & Remediation

5.1 SQL Injection (High Severity)

- **The Root Cause:** The application constructs dynamic SQL queries using string concatenation with user-supplied input from the email login field [cite: 1].
- **Impact:** An attacker can input `' OR 1=1 --` into the login fields, causing the database to evaluate the query statement as always true. This allows unauthorized access to administrative or private accounts without knowing the actual password [cite: 1].
- **Remediation:** Implement Parameterized Queries (Prepared Statements). This separates user data from the execution code logic, ensuring that inputs are treated strictly as data literals and never executed as database commands [cite: 1].

5.2 Cross-Site Scripting (Medium Severity)

- **The Root Cause:** The search mechanism echoes the searched string directly back onto the webpage DOM without proper escaping or encoding [cite: 1].
- **Impact:** An attacker can craft a malicious URL containing javascript payload like `<script>alert(document.cookie)</script>`. Anyone clicking that URL will execute the script within their active browser session [cite: 1].
- **Remediation:** Utilize Context-Aware Output Encoding (e.g., converting `<` to `<` and `>` to `>`). This ensures the browser displays the string as plain text rather than executing it as an active script block [cite: 1].

6. Conclusion & Recommendations

The security audit conducted via OWASP ZAP conclusively proves that the target application has blatant architectural vulnerabilities. Because security validation controls are missing at both the client and server levels, malicious users can compromise user accounts and leak database structures easily [cite: 1].

Strategic Roadmap: [cite: 1]

1. **Source Code Review:** Conduct an immediate static code analysis targeting backend input authentication points [cite: 1].
2. **Framework Upgrades:** Leverage modern application frameworks that have built-in defenses against XSS and SQL Injection [cite: 1].
3. **Continuous Testing:** Integrate automated security scanning tools (like OWASP ZAP) into CI/CD pipelines to catch vulnerability regressions before code reaches production systems [cite: 1].

7. SQL Injection Solution & Code Implementation

To completely remediate the SQL Injection vulnerability identified in the login route (`routes/login.ts`), vulnerable string concatenation must be replaced with parameterized queries or secure Object-Relational Mapping (ORM) methods. Below is the implementation demonstrating the insecure approach versus the secure fix:

Vulnerable Code Implementation (Before Fix)

```
// ❌ VULNERABLE: Direct string concatenation of user input
const query = "SELECT * FROM users WHERE email = '" + req.body.email + "' AND password = '" + req.body.password + "'";
db.execute(query);
```

Secure Code Implementation (After Fix)

```
// ✅ SECURE: Using Parameterized Query / Prepared Statements
const query = "SELECT * FROM users WHERE email = ? AND password = ?";
db.execute(query, [req.body.email, req.body.password], (err, results) => {
  if (err) {
    return res.status(500).json({ error: "Database error" });
  }
  if (results.length > 0) {
    return res.status(200).json({ message: "Login successful", user: results[0] });
  } else {
    return res.status(401).json({ error: "Invalid credentials" });
  }
});
```